
CPR E 4890: Computer Networking and Data Communications

Lab Experiment #3 – UDP Socket Programming

(Total Points: 100)

Objective

To use UDP sockets to write a program to implement network impairments (i.e., loss and delay), and study this effect on a video stream.

Pre-Lab

- Review the socket programming materials.
- Write the code to create a UDP socket and bind it to the IP address of the machine your program is running on, and to a given port number.

Lab Expectations

Work through the lab and let the TA know if you have any questions. After the lab, write up a lab report. Be sure to:

- 1) Attend the lab. (5 points)
- 2) Summarize what you learned in a few paragraphs. (15 points)
- 3) Include your answers to the **two exercises** stated in the “Exercises” section below. (30 points in total, with 15 points each)
- 4) Include your **well-commented** code and **demo** your code to the TA. (50 points)
 - See the detailed point breakdown in the “Problem Description” section.

Demonstration Policy

This lab will require you to **demo** your code to the TA. Review the lab manual carefully to ensure you have completed all demonstration requirements. Labs may be demonstrated during:

- **this lab section;**
- **the TA office hours;**
- **or, the first hour of the next lab (Lab #4).**

Problem Description

In this lab experiment, you are required to implement a UDP program that **forwards UDP packets from a specified source to a specified destination while intentionally introducing packet loss**. Figure 1 illustrates this setup.

- The program will run on a Linux machine and will accept five parameters: server IP, server port, destination IP, destination port, and loss rate. (10 points)
- The program will listen on the port number specified by the server port parameter and will accept packets for the address specified by the server IP parameter. (10 points)
- Packets will be forwarded to the machine and port specified by the destination IP and destination port parameters, respectively. (20 points)
- The program will decide whether to forward each received packet based on the specified loss rate. (10 points)

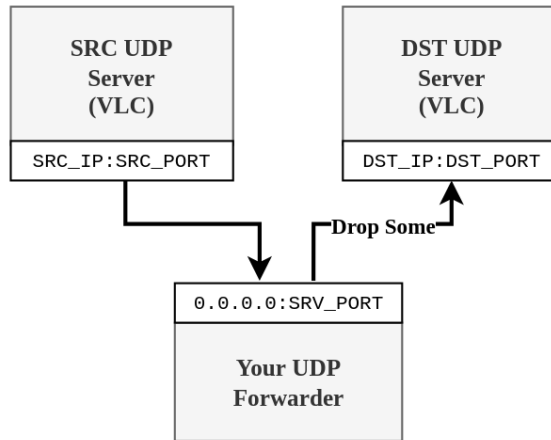


Figure 1. UDP Forwarder Architecture - Your program receives packets from the source VLC player and forwards them to the destination VLC player, dropping packets according to the specified loss rate.

Procedure

- Write the program in C under Linux.
 - Edit your program using any of the editors available under UNIX (e.g., VSCode, pico, vi, emacs, etc.).
 - A skeleton project has been provided, which includes a README.md file with instructions for getting started using the makefile.
- Compile your program using `gcc`, or the makefile provided in the skeleton project by using the command `make`
- Run your program by typing the full path to the compiled executable along with its command-line arguments. For example, if the executable is named `udp_forwarder`, then type:
`./udp_forwarder <SERVER_IP> <SERVER_PORT> <DESTINATION_IP> <DESTINATION_PORT> <LOSS_RATE>`
- `<SERVER_IP>` and `<SERVER_PORT>` are the IP and port your UDP forwarding server will receive data on. These are also the IP and port which the source VLC player will stream video to.
- `<DESTINATION_IP>` and `<DESTINATION_PORT>` are where you want to dispatch the video, which is discussed in Step 4 of the “**Testing Your Program**” section.
- `<LOSS_RATE>` is the number of packets out of 1000 that are dropped by your program. For example, a loss rate of 50 means that 50 out of every 1000 packets (5%) will be dropped. (Hint: you may implement a loss rate by comparing a random number with the loss rate. For example, if the loss rate is 50, generate a random number between 0 and 1000, and if it is smaller than 50, drop the packet; otherwise, forward the packet.)
- The source port and destination port parameters should use separate UDP port numbers chosen between 1024 and 65535.

Note: Due to lab network limitations, use **127.0.0.1 (localhost)** for all IP addresses when testing your program. This means both `SERVER_IP` and `DESTINATION_IP` should be `127.0.0.1`, but `SERVER_PORT` and `DESTINATION_PORT` must be different.

Testing Your Program

To test your program, you will be using a media player called “**VLC media player**.” VLC media player allows for playing multimedia files of various formats, while providing the ability to stream the file being played to a remote machine specified by an IP address and a port number. Additionally, VLC can display media available through a network stream.

1. Identify three machines on which you will run the **source stream**, the **destination player**, and **your forwarding server**. For lab testing, use 127.0.0.1 as the IP address for all three components (they will run on the same machine using different ports).
2. To demonstrate and test your program, you will need to forward packets between a source VLC stream and a destination VLC viewer. In other words, your single program will act as a server (to accept UDP packets from the source) and a client (to forward UDP packets to the destination). **Your program only needs one UDP socket**. This socket will bind to **SERVER_IP:SERVER_PORT** to receive packets, then use `sendto()` to forward packets to **DESTINATION_IP:DESTINATION_PORT**.
3. Start the source VLC program using the below command format:

```
cvlc --repeat test.mp4 --sout \
'#standard{access=udp,mux=ts,dst= <SERVER_IP>:<SERVER_PORT>}'
```

4. To watch the video data sent by the above program at the destination, you would start a new terminal on the destination computer and enter the following command, where **DESTINATION_PORT** shall use a **different** port number from **SERVER_PORT**. Do not forget the @ symbol!

```
vlc -vvv udp://@<DESTINATION_IP>:<DESTINATION_PORT>
```

Note: *DESTINATION_PORT must be different from SERVER_PORT. If they are the same, the destination VLC will bind to the port before your forwarder can start, causing your program to fail.*

5. Start your program with the following parameters: the server IP is the IP address your server will bind to, the server port is the port number entered in Step 3 above, the destination IP is the IP address of the destination VLC player, the destination port is the port number entered in Step 4, and the loss rate is any value from 0 to 1000. For example:

```
./udp_forwarder 127.0.0.1 7000 127.0.0.1 8000 50
```

6. Loss rate of 0 should produce lossless video at the destination, loss rate of 1 should drop 1 out of every 1000 packets (0.1% loss), loss rate of 2 should drop 2 out of every 1000 packets (0.2% loss), etc.
7. Submit a .zip of your source code **alongside** your lab report to the TA for grading.

Exercises

- 1) What is the effect of increasing the loss rate on the video quality? **(15 points)**
- 2) To counter the effect of network loss, someone suggests using TCP instead of UDP for multimedia communication. Do you agree? Why? Why not? (*Hint: answer the question in terms of buffer space, jitter, and retransmission time.*) **(15 points)**

Notes

- If any information is missing, you may make reasonable assumptions and state them in your solution.
- Refer to the “**Procedure**” section above for general guidelines on how to write and run your program.
- VLC media player has been pre-installed on all lab machines.
- A test video file (test.mp4) is provided in the skeleton project.
- If you are having issues launching VLC, use the following command:
`MALLOC_CHECK_=1 vlc`